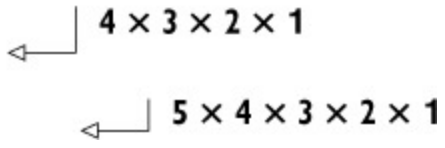
```
factorial = (n) ->
  if n is 0 then 1
  else
    n * (factorial n - 1)

factorial 0
# 1

factorial 4
# 24
factorial 5
# 120
```



The value of `factorial 5` is actually the value of `factorial 4` multiplied by 5. This means that once `factorial 4` has been evaluated, then 4 × 3 × 2 × 1 has already been done once, so `factorial 5` could use the value already worked out instead of working the whole thing out again. Memoization is useful to avoid repeating the same evaluation. There's something else interesting about the definition of `factorial`: it invokes itself. This is called *recursion*, and like memoization it's useful not only for things like factorials but also for problems you actually have.

6.3.3. Recursion

Imagine now that the database is having more problems, and half of all requests to the users database result in a timeout. Data is retrieved from the database by invoking `users.get` with a callback function. When the request succeeds, you get data back as the second argument to the callback. You can see this happening by logging to the console:

```
logUserDataFor = (user) ->
  users.get user, (error, data) ->
    if error then console.log 'An error occurred'
    else console.log 'Got the data'

logUserDataFor 'fred'
# 'Got the data'
```

When a timeout occurs, the callback is invoked with an error as the first argument: